

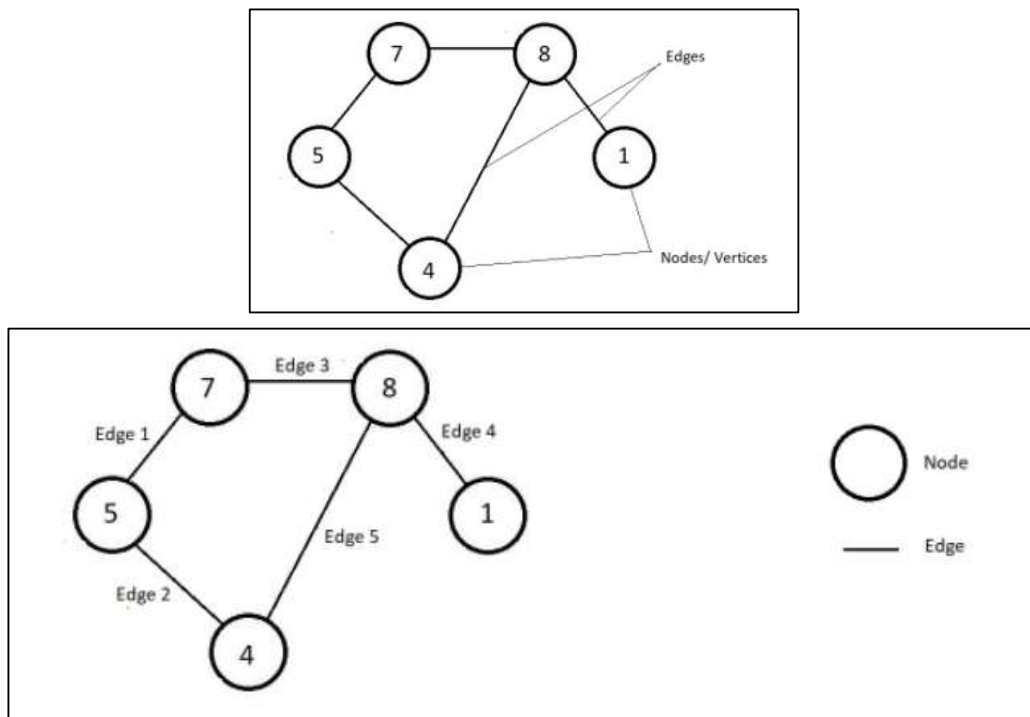
CHAPTER 8

Graph Algorithm

A graph (G) is a data structure (V, E) that consists of

- A collection of vertices V
- A collection of edges E, represented as ordered pairs of vertices (u,v)

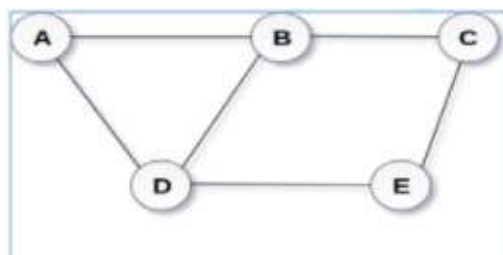
A Graph G can be defined as an ordered set of $G(V,E)$ where $V(G)$ represents the set of vertices and $E(G)$ represents the set of edges which are used to connect the vertices. It is a non-linear data structure.



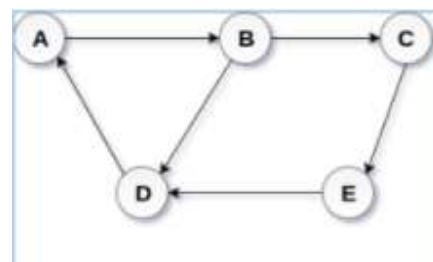
Directed Graph : Directed graphs are graphs having each of its edges directed.

Undirected Graph : Undirected graphs are graphs having each of its edges undirected.

Following figure shows undirected and directed Graph $G(V,E)$ with 5 vertices (A, B, C, D, E) and six edges {(A,B), (B,C), (C,E), (E,D), (D,B), (D,A)} :



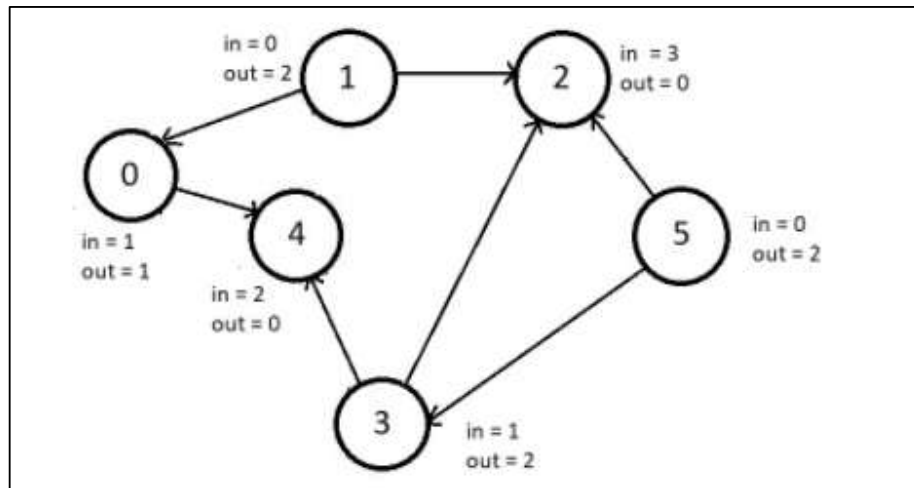
Undirected Graph



Directed Graph

Indegree and outdegree of a node :

Indegree of a node is the number of edges coming to the node, and outdegree of a node is the number of edges originating from that node.



Representation of Graphs:

Any representation should basically be able to store the nodes of a graph and their connections between them. And this can be accomplished in so many ways but primarily the most used way to represent a graph is,

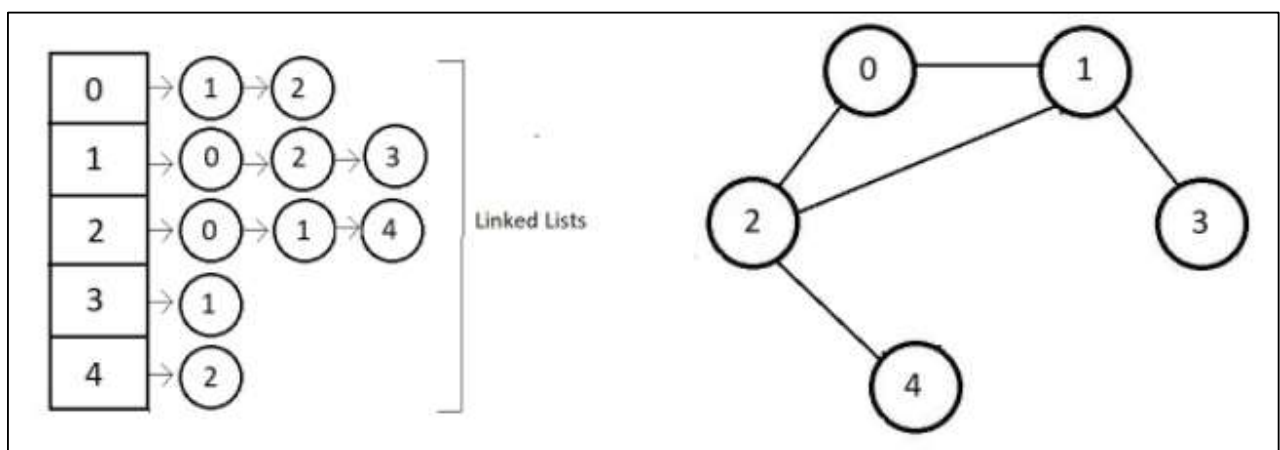
Adjacency List - Mark the nodes with their neighbours

Adjacency Matrix - $A_{ij} = 1$, if there is an edge between i and j , 0 otherwise.

Edge Set - Store the pair of nodes/vertices connected with an edge. Example: $\{(0, 1), (0, 4), (1, 4)\}$.

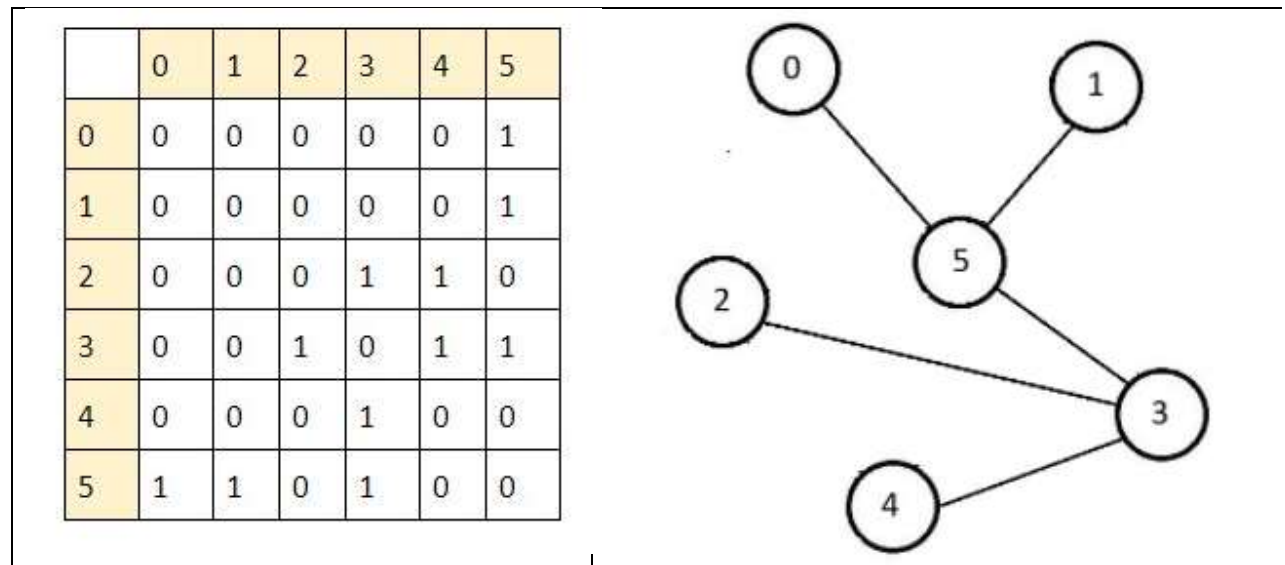
Adjacency List

In this method of representation of graphs, we store the nodes, along with the list of their neighbors. Basically, we maintain a list of all the nodes, and along with it, we store the list of nodes a particular node is connected with.



Adjacency Matrix

Adjacency matrix is another method of representation of graphs, where we represent our graph in the form of a matrix where cells are either filled with 0 or 1. Let's call the cell falling on the intersection of i th row and j th column be A_{ij} , then the cell would be filled with 1 if there is an edge between node i and j , otherwise, the cell would be filled with a 0.



Graph Traversal

Graph traversal refers to the process of visiting (checking and/or updating) each vertex (node) in a graph. A smaller graph seems relatively easy to traverse. But when it comes to traversing a graph with a huge number of nodes, the process needs to be automated. Doing things manually increases the chances of missing some vertices or so. And visiting nodes of a graph becomes important when you need to change something for some nodes, or just need to retrieve the value present there or something else. And this is where our graph traversing algorithms come to the rescue.

There are two algorithms that are used for traversing a graph. They are:

- Breadth-First Search (BFS)
- Depth First Search (DFS)

Breadth First Search(BFS)

- Breadth First Search or BFS is a graph traversal algorithm.
- It is used for traversing or searching a graph in a systematic fashion.
- BFS uses a strategy that searches in the graph in breadth first manner whenever possible.
- Queue data structure is used in the implementation of breadth first search.

Remember !

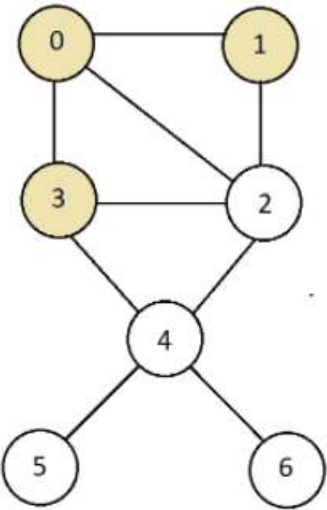
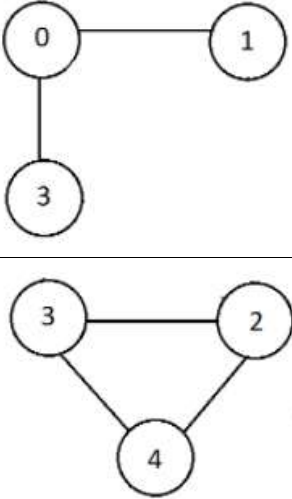
- ✓ We can start with any vertex.
- ✓ There can be multiple Breadth-First Search results for a given graph
- ✓ The order of visiting the vertices can be anything.

Depth First Search(DFS)

- Depth First Search or DFS is a graph traversal algorithm.
- It is used for traversing or searching a graph in a systematic fashion.
- DFS uses a strategy that searches “deeper” in the graph whenever possible.
- Stack data structure is used in the implementation of depth first search.

Subgraph

A subgraph of a graph G is a graph whose vertices and edges are subsets of the original graph G. It means that if we consider the graph G illustrated below, then its subgraphs could be the graph S.

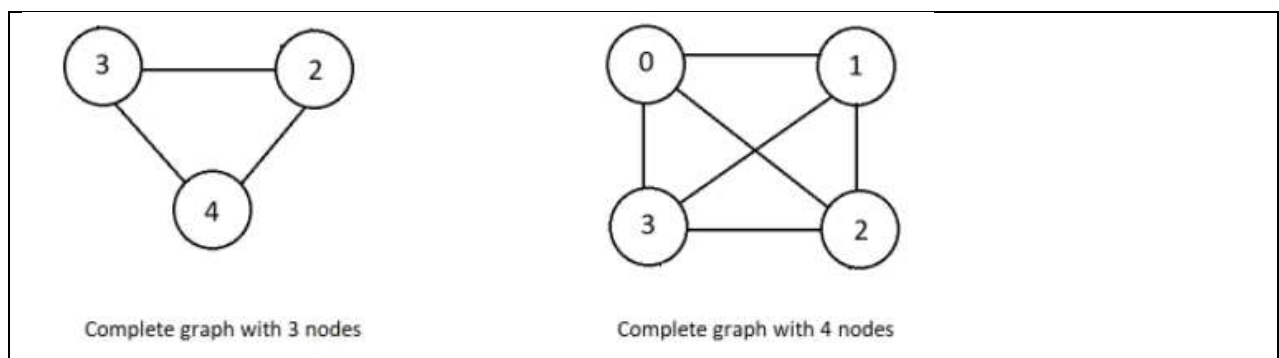
Graph (G)	Subgraphs (S)
	

Connected Graph

A connected graph, as the word connected suggests, is a graph that is connected in the sense of a topological space, i.e., that there is a path from any point to any other point in the graph. And the graph which is not connected is said to be disconnected.

Complete Graph

A complete graph is a simple undirected graph in which every pair of distinct vertices is connected by a unique edge.

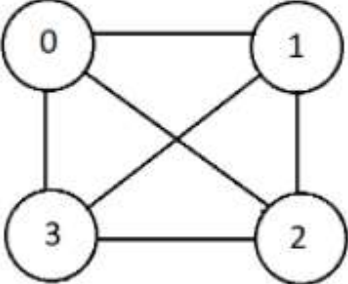
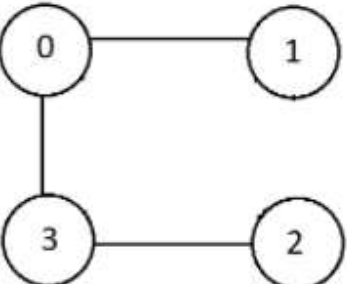
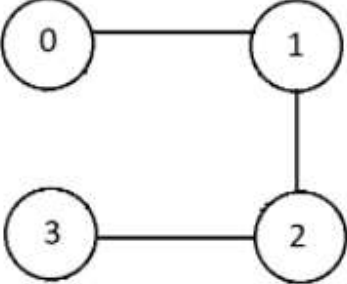
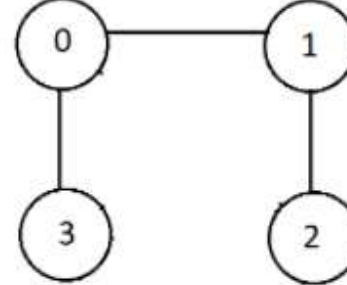


Spanning Tree

A **connected subgraph** 'S' of a graph G (V, E) is said to be a spanning tree of the graph if and only if:

- All the vertices of G are present in S,
- No. of edges in S should be $|V|-1$, where $|V|$ represents the number of vertices.

That is, for a subgraph of a graph to be called a spanning tree of that graph, it should have all vertices of the original graph and must have exactly $|V|-1$ edges, where $|V|$ represents the number of vertices and the graph should be connected.

Graph (G)	Spanning Tree	
		
		

A complete graph has n^{n-2} spanning trees where n represents the number of vertices in the graph. This complete graph has 4 vertices, hence the number of spanning trees it can have is, 4 raised to the power $(4-2)$, i.e., 4^2 which is 16.

Is graph subgraph of G?

Is graph connected?

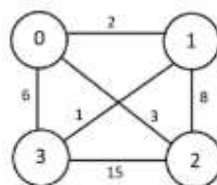
Are all vertices of Graph G present in graph S?

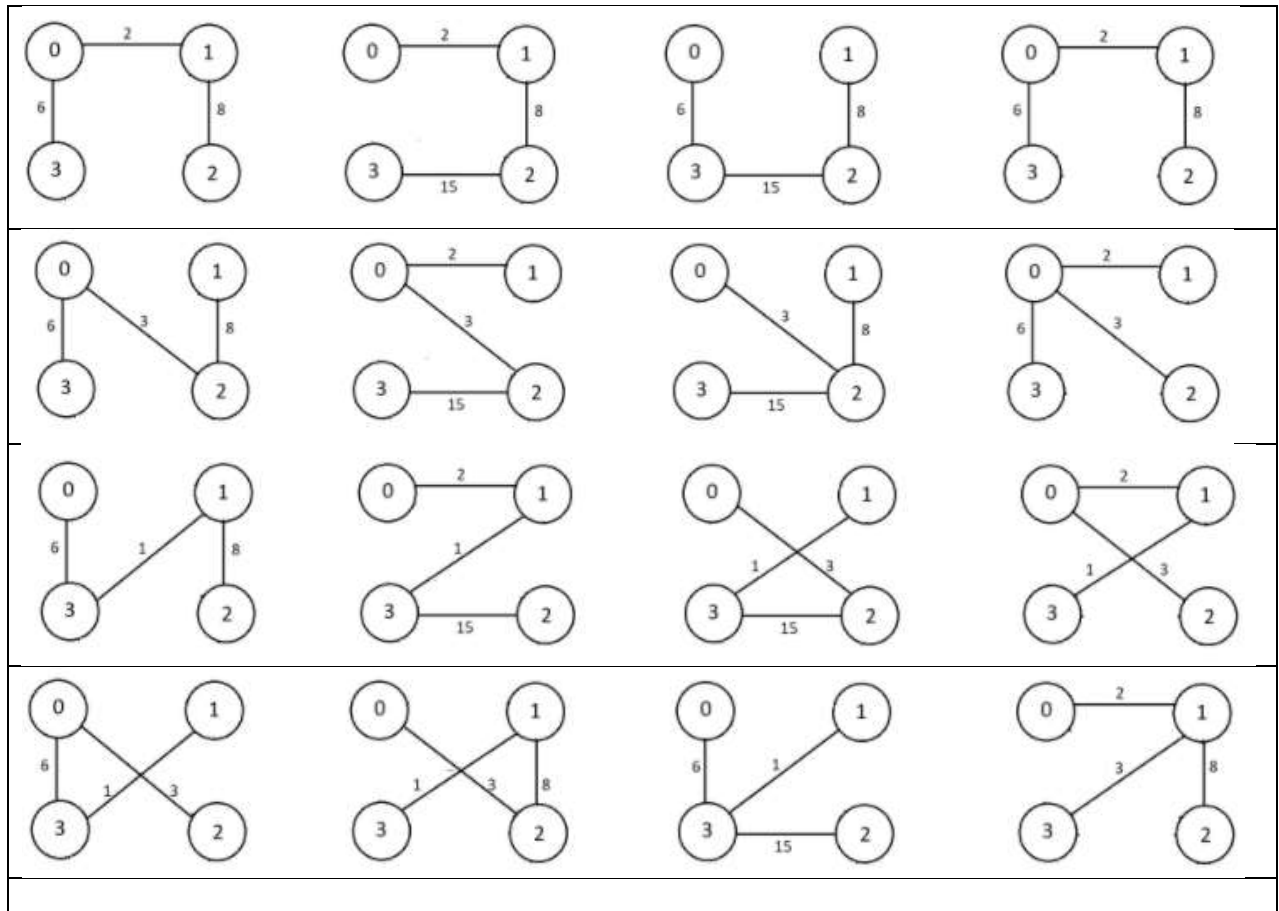
Does the number of edges in S is less than that of G by 1?

Minimum spanning tree:

A Minimum Spanning tree, abbreviated as MST, is the spanning tree having the minimum cost.

Finding the minimum spanning tree of :





Algorithm of Finding Minimum Spanning Tree

A) Prim's Algorithm

- Prim's Algorithm is a famous greedy algorithm.
- It is used for finding the Minimum Spanning Tree (MST) of a given graph.
- To apply Prim's algorithm, the given graph must be weighted, connected and undirected.

Steps 1 :

- Randomly choose any vertex.
- The vertex connecting to the edge having least weight is usually selected.

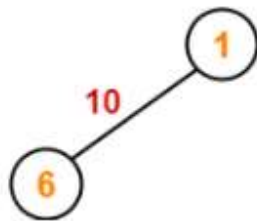
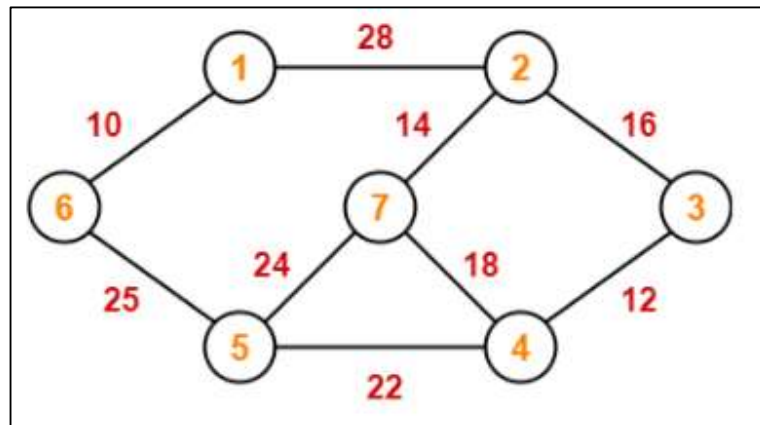
Step 2 :

- Find all the edges that connect the tree to new vertices.
- Find the least weight edge among those edges and include it in the existing tree.
- If including that edge creates a cycle, then reject that edge and look for the next least weight edge.

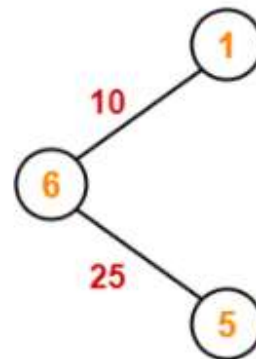
Step 3 :

- Keep repeating Step 2 until all the vertices are included and Minimum Spanning Tree (MST) is obtained.

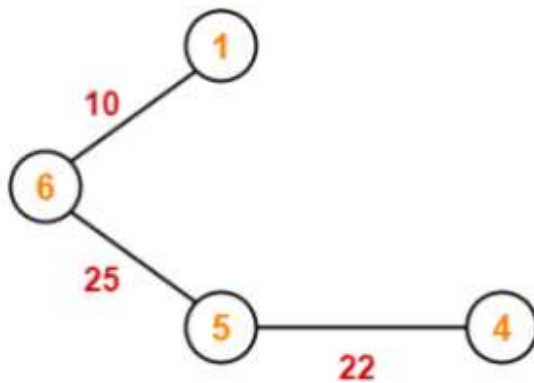
For Example :



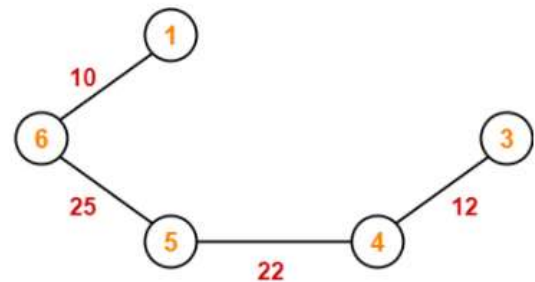
(I)



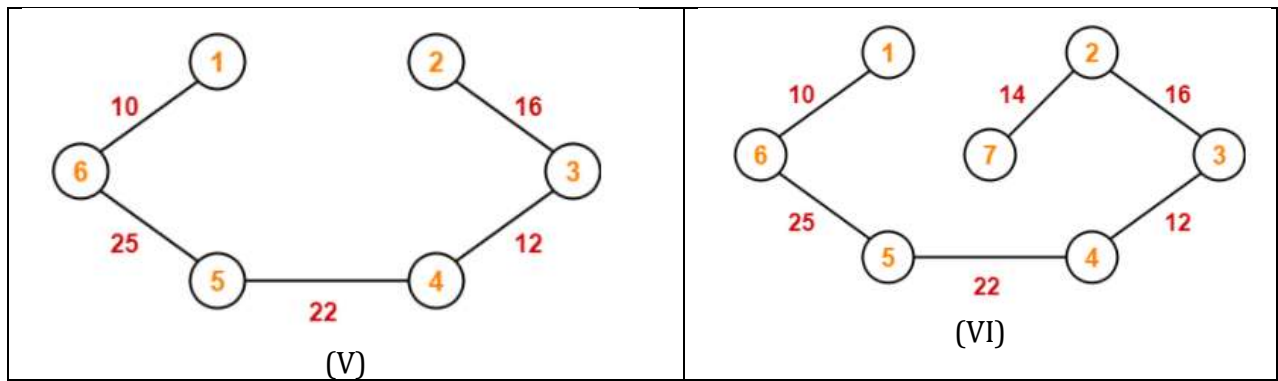
(II)



(III)



(IV)



Since all the vertices have been included in the MST, so we stop.

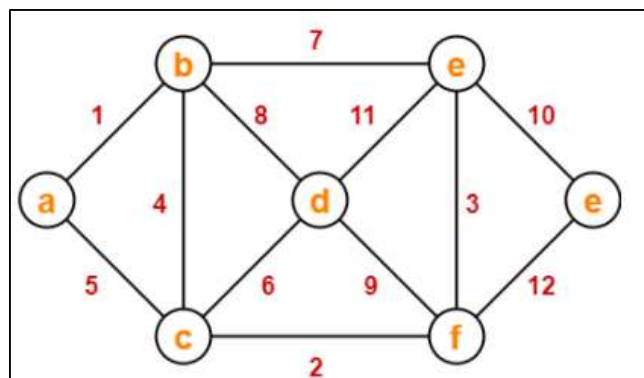
Now, Cost of Minimum Spanning Tree

= Sum of all edge weights

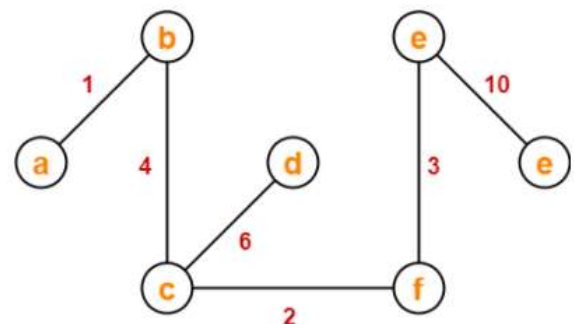
= $10 + 25 + 22 + 12 + 16 + 14$

= 99 units

Q) Using Prim's algorithm find the MST of following graph



Answer = 26 units



B) Kruskal's Algorithm

- Kruskal's Algorithm is a famous greedy algorithm.
- It is used for finding the Minimum Spanning Tree (MST) of a given graph.
- To apply Kruskal's algorithm, the given graph must be weighted, connected and undirected.

Steps 1 :

- Sort all the edges from low weight to high weight.

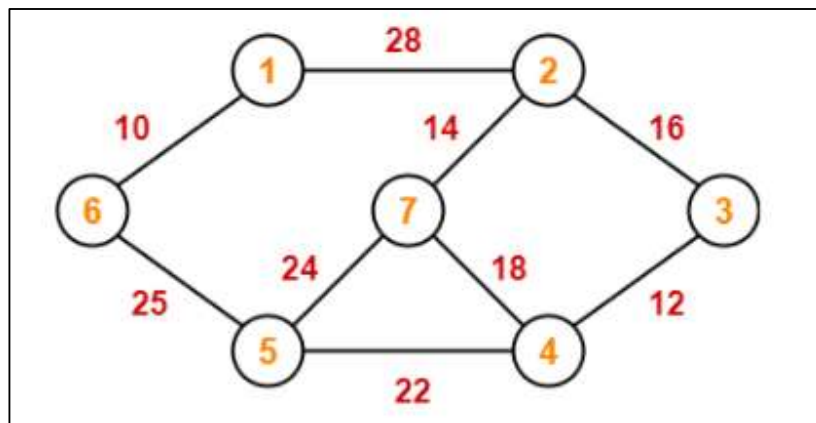
Step 2 :

- Take the edge with the lowest weight and use it to connect the vertices of graph.
- If adding an edge creates a cycle, then reject that edge and go for the next least weight edge.

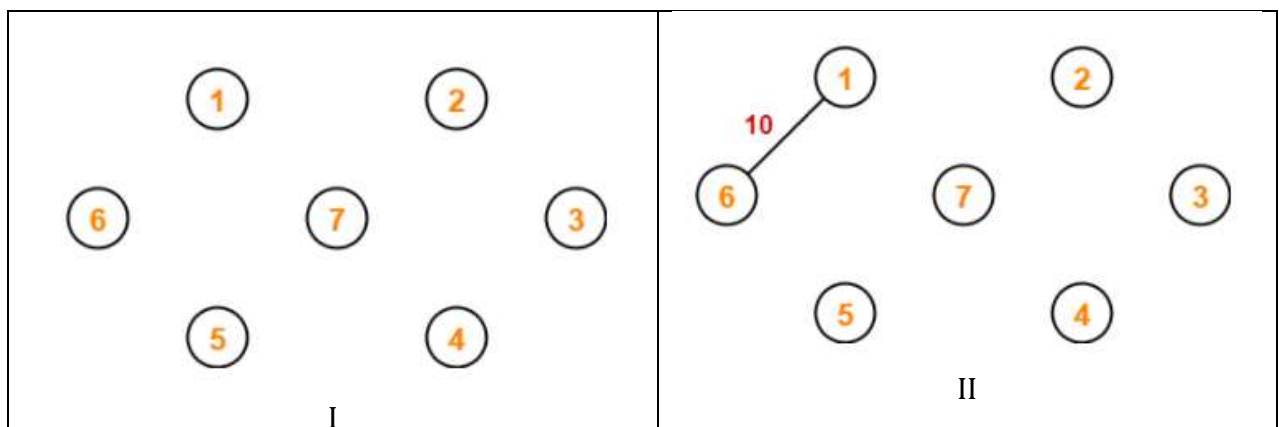
Step 3 :

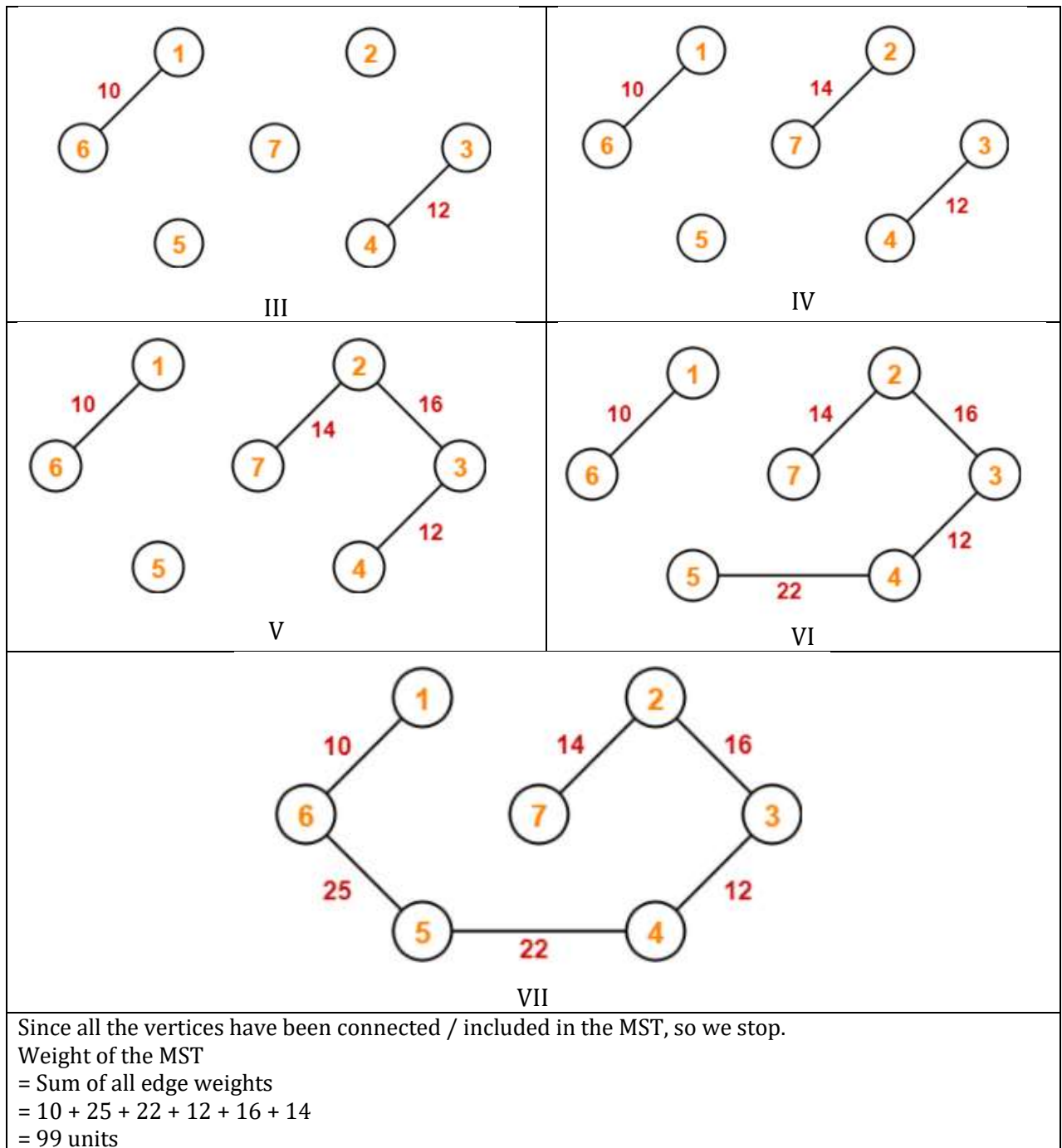
- Keep adding edges until all the vertices are connected and a Minimum Spanning Tree (MST) is obtained.

For Example :



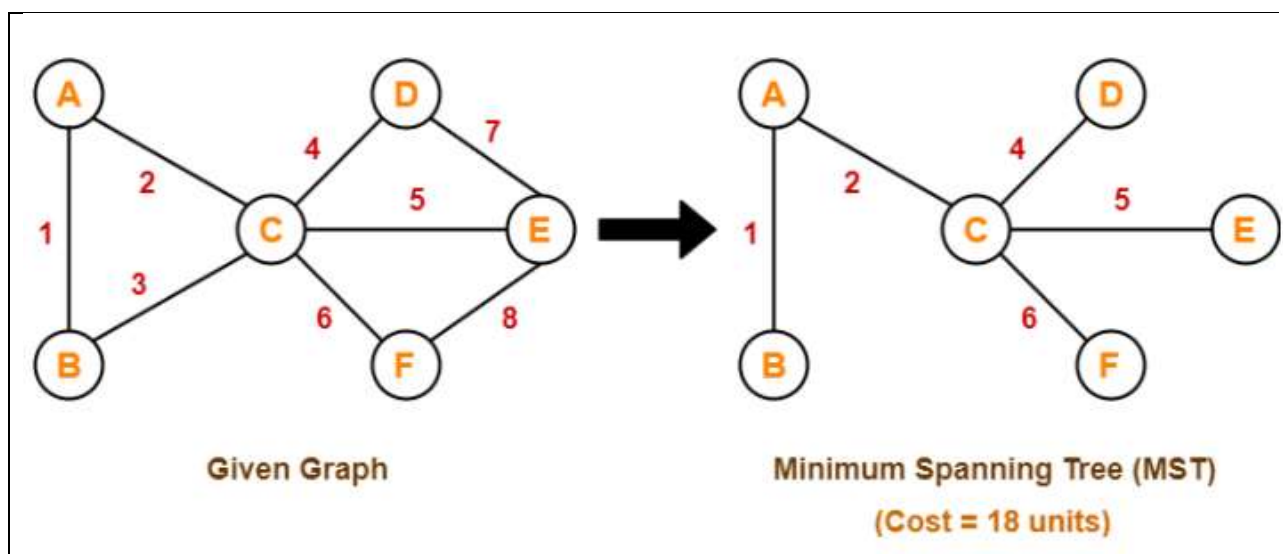
Solution :

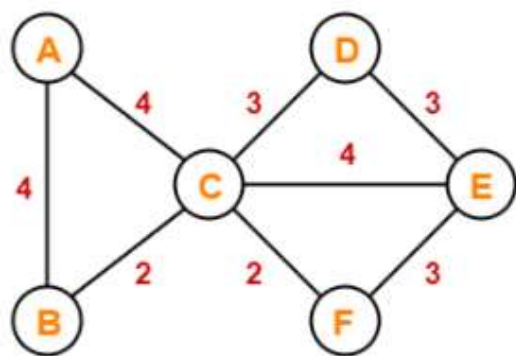




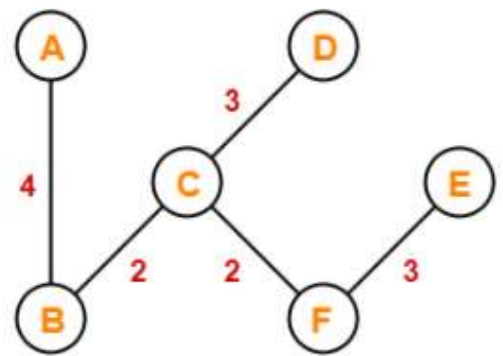
- Prim's and Kruskal's Algorithm are the famous greedy algorithms.
- They are used for finding the Minimum Spanning Tree (MST) of a given graph.
- To apply these algorithms, the given graph must be weighted, connected and undirected.
- If all the edge weights are distinct, then both the algorithms are guaranteed to find the same MST.
- If all the edge weights are not distinct, then both the algorithms may not always produce the same MST. However, cost of both the MSTs would always be same in both the cases.

Prim's Algorithm	Kruskal's Algorithm
The tree that we are making or growing always remains connected.	The tree that we are making or growing usually remains disconnected.
Prim's Algorithm grows a solution from a random vertex by adding the next cheapest vertex to the existing tree.	Kruskal's Algorithm grows a solution from the cheapest edge by adding the next cheapest edge to the existing tree / forest.
Prim's Algorithm is faster for dense graphs.	Kruskal's Algorithm is faster for sparse graphs.

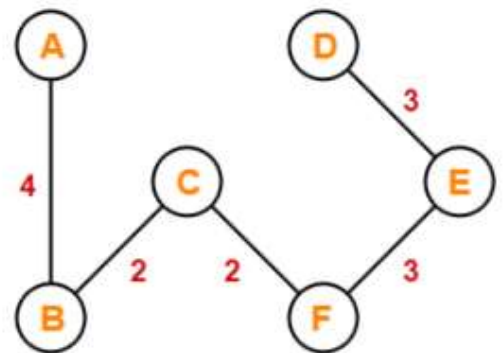




Given Graph



Result from Prim's Algorithm
(Cost = 14 units)



Result from Kruskal's Algorithm
(Cost = 14 units)

Shortest Path Algorithm

Shortest path problem is a problem of finding the shortest path(s) between vertices of a given graph. Shortest path between two vertices is a path that has the least cost as compared to all other existing paths. It has applications in Google Maps, Road Networks, Logistics Research, etc.

Various types of shortest path problem are-

- Single-pair shortest path problem
- Single-source shortest path problem
- Single-destination shortest path problem
- All pairs shortest path problem

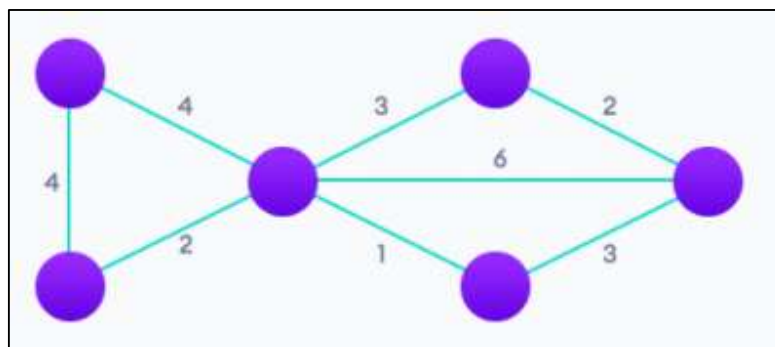
Dijkstra's Algorithm is a famous algorithm adapted for solving single-destination shortest path problem.

Dijkstra algorithm

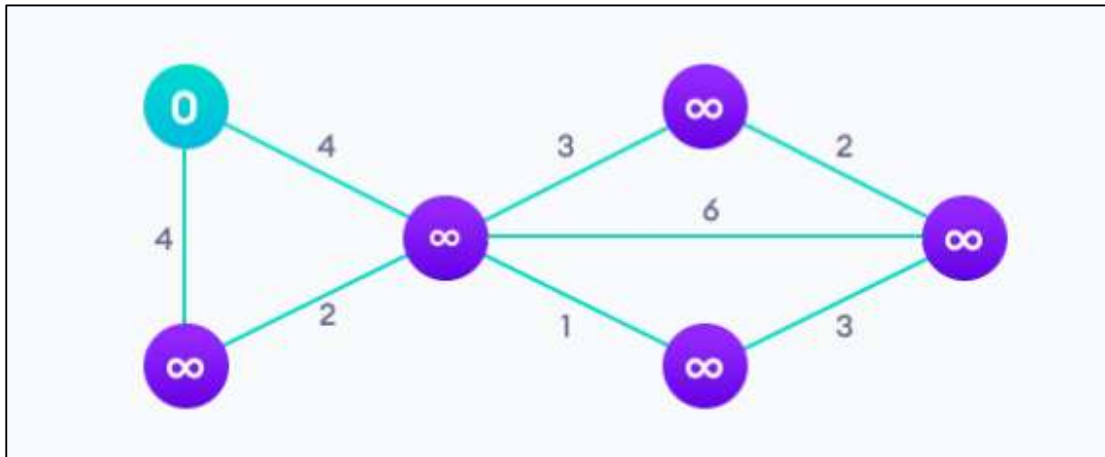
- Dijkstra algorithm works only for connected graphs.
- Dijkstra algorithm works only for those graphs that do not contain any negative weight edge.
- The actual Dijkstra algorithm does not output the shortest paths.
- It only provides the value or cost of the shortest paths.
- By making minor modifications in the actual algorithm, the shortest paths can be easily obtained.
- Dijkstra algorithm works for directed as well as undirected graphs

For example :

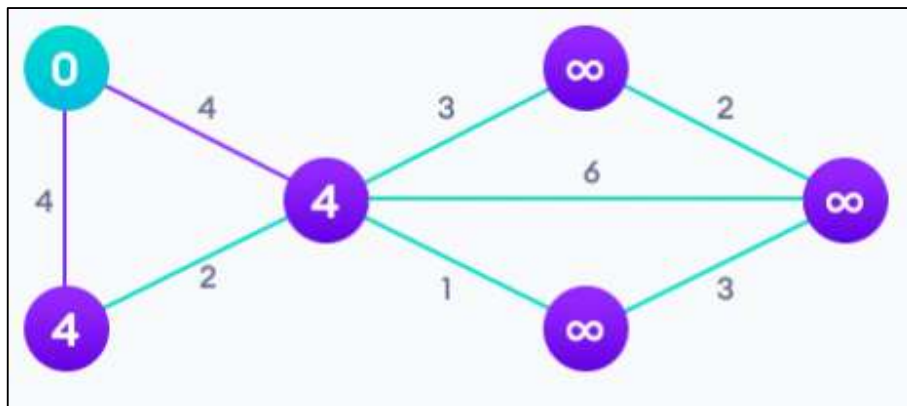
We start with a weighted graph as shown in figure below:



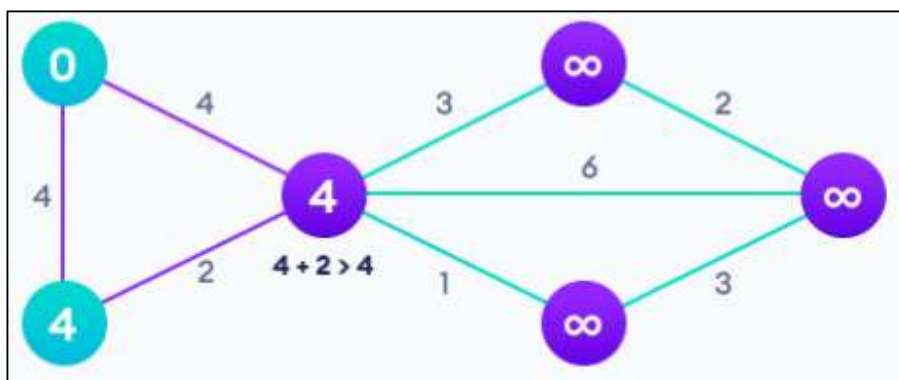
Choose a starting vertex and assign its path value as '0' and other path value as ∞ .



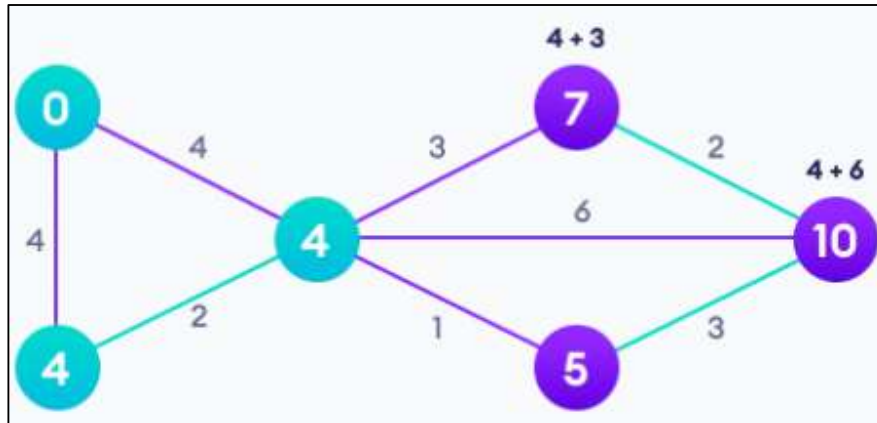
Go to each vertex and update its path length



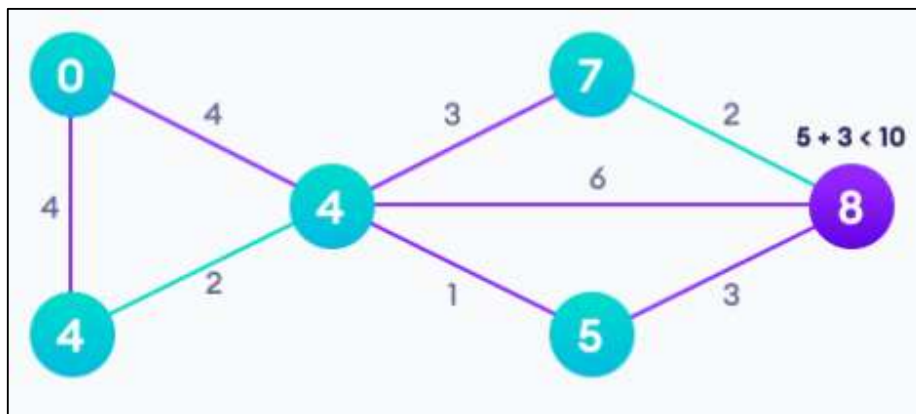
If the path length of the adjacent vertex is lesser than new path length, don't update it



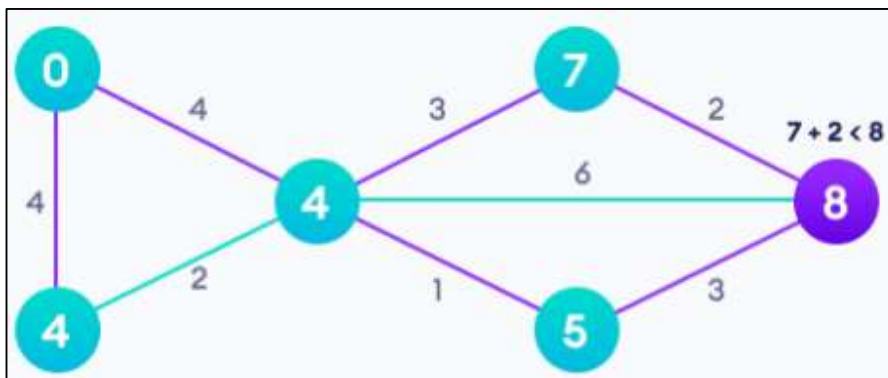
Avoid updating path lengths of already visited vertices



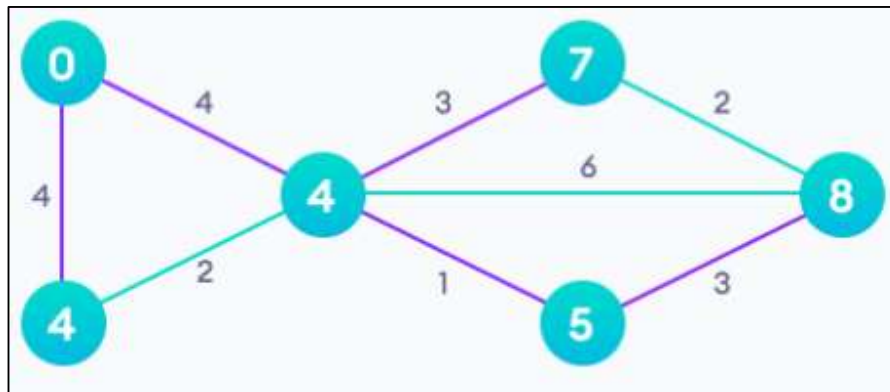
After each iteration, we pick the unvisited vertex with the least path length. So we choose 5 before 7.



Notice how the rightmost vertex has its path length updated twice



Repeat until all the vertices have been visited



Refer class note for more examples.

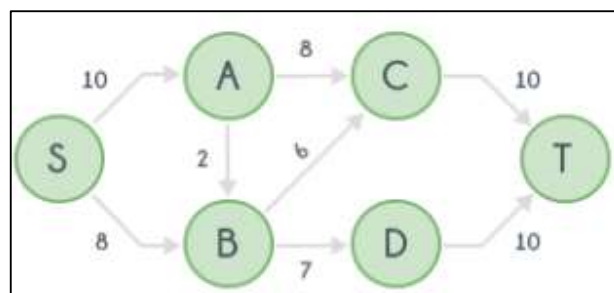
Network Flow Problem

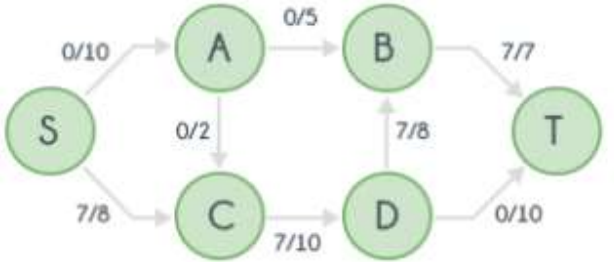
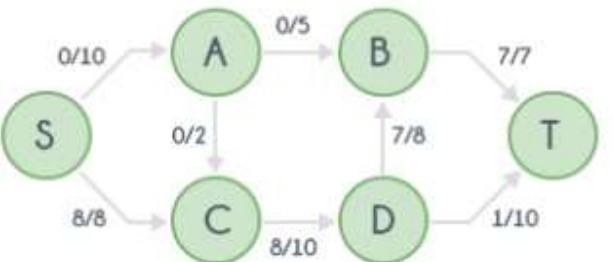
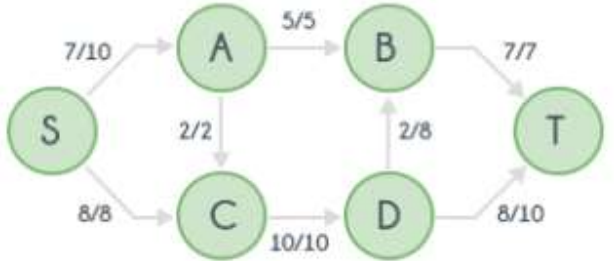
- A network is a directed graph.
- Edge represents pipes that carry flow.
- Each edge has a maximum capacity.
- A source node 'S' in which flow arrives.
- A sink/destination node 't' out of which flow leaves.

Network Flow problem is to maximize the total amount of flow from s to t subject to two constraints;

- Flow on edge should not exceed capacity (c).
- For every node $v \neq s$, incoming flow is equal to outgoing flow.

For example :

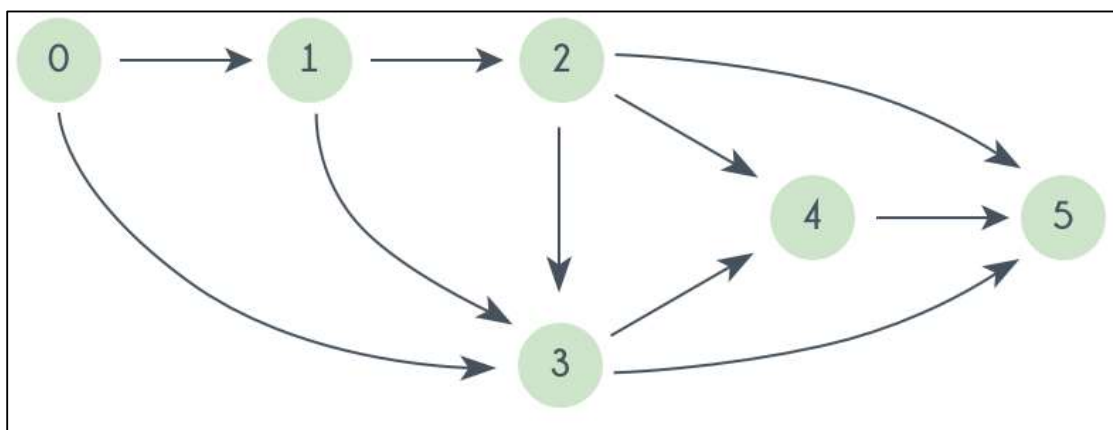


	Augmented Path : S-C-D-B-T Bottleneck Capacity : 7
	Augmented Path : S-C-D-T Bottleneck Capacity : 1
	Augmented Path : S-A-C-D-T Bottleneck Capacity : 2
Repeat the process until there is a path left.	

Topological Sort

It is the linear ordering of a Directed Acyclic Graph (DAG) such that for every directed edge uv , u comes before c in the ordering.

Following is the example of topological sorting.



in_degree <table><tr><td>0</td><td>1</td><td>2</td><td>2</td><td>2</td><td>3</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table> Initially the indegree of '0' is 0 and T is empty. So we remove 0 from the queue and put in in T.	0	1	2	2	2	3	0	1	2	3	4	5	T:
0	1	2	2	2	3								
0	1	2	3	4	5								
in_degree <table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>2</td><td>3</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table> 0 is directly connected to 1 and 3. So we decrease the degree of 1 and 3.	0	0	1	1	2	3	0	1	2	3	4	5	T: 0
0	0	1	1	2	3								
0	1	2	3	4	5								
in_degree <table><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>3</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table> Here we remove 1 from the queue and append to T. Doing this will decrease the degree of 2 to 0 .	0	0	0	1	1	3	0	1	2	3	4	5	T: 0 1
0	0	0	1	1	3								
0	1	2	3	4	5								
in_degree <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>2</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	0	0	0	1	2	0	1	2	3	4	5	T: 0 1 2
0	0	0	0	1	2								
0	1	2	3	4	5								
in_degree <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	0	0	0	0	1	0	1	2	3	4	5	T: 0 1 2 3
0	0	0	0	0	1								
0	1	2	3	4	5								
in_degree <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	0	0	0	0	0	0	1	2	3	4	5	T: 0 1 2 3 4
0	0	0	0	0	0								
0	1	2	3	4	5								
in_degree <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	0	0	0	0	0	0	1	2	3	4	5	T: 0 1 2 3 4 5
0	0	0	0	0	0								
0	1	2	3	4	5								

Final Topological Sorting is : 0 1 2 3 4 5